

Module 20: Indexes

Indexes are database objects that improve the speed of data retrieval operations. They work similarly to the index of a book, allowing SQL Server to locate data quickly without scanning the entire table.



Introduction

Imagine a table containing:

10,000,000 Employees

You execute:

```
SELECT *  
FROM Employees  
WHERE EmployeeID = 1001;
```

Without an index:

SQL may scan millions of rows.

With an index:

SQL can directly locate the required row.

This is why indexes are critical for database performance.



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand indexes
- ✓ Understand clustered indexes
- ✓ Understand non-clustered indexes
- ✓ Create indexes

- ✓ Drop indexes
- ✓ Use composite indexes
- ✓ Understand index performance
- ✓ Optimize SQL queries



Table of Contents

1. What is an Index?
2. Why Indexes are Needed
3. How Indexes Work
4. Clustered Index
5. Non-Clustered Index
6. Unique Index
7. Composite Index
8. Creating Indexes
9. Dropping Indexes
10. Index Advantages
11. Index Disadvantages
12. Index Maintenance
13. Execution Plans
14. Best Practices
15. Interview Questions
16. Business Analytics Examples
17. Summary
18. Practice Questions



1 What is an Index?

An index is a special lookup structure that helps SQL Server find rows faster.

Book Example

Without index:

Read every page.

With index:

Find topic in index.
Jump directly to page.

Databases use the same concept.

Example

Table:

EmployeeID	EmployeeName
1	Kunal
2	Rahul
3	Aman

Query:

```
SELECT *  
FROM Employees  
WHERE EmployeeID = 3;
```

Index helps SQL locate:

```
EmployeeID = 3
```

quickly.

Why Indexes Matter

Used for:

- Faster Searches
- Faster Joins
- Faster Sorting
- Faster Reporting
- Better Dashboard Performance

2 Why Indexes are Needed

Consider:

1 Million Rows

Without Index

Table Scan

SQL checks every row.

With Index

Index Seek

SQL directly finds matching rows.

Performance Difference

Example:

Without Index

5 Seconds

With Index

50 Milliseconds

3 How Indexes Work

Most database systems use:

B-Tree Structure

Structure:

```
Root
|
├── Branch
|
└── Leaf Nodes
```

SQL navigates the tree.

Instead of reading:

Every Row

it jumps directly to required data.

Simplified Example

```
EmployeeID

1
2
3
4
5
...
```

Index stores ordered references.

Benefits

Fast Search
Fast Sorting
Fast Filtering

Clustered Index

Most important index type.

Definition

A clustered index determines the physical order of data inside a table.

Key Rule

Only ONE Clustered Index Per Table

Why?

A table can only be physically sorted one way.

Example

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(100)
);
```

SQL Server automatically creates:

Clustered Index

on Primary Key by default.

Data Storage

```
1  
2  
3  
4  
5
```

stored physically in order.

Best For

```
Primary Keys  
Date Columns  
Frequently Sorted Data
```

5 Non-Clustered Index

Most commonly created manually.

Definition

Stores:

```
Indexed Values  
+  
Pointer to Actual Data
```

Example

```
CREATE NONCLUSTERED INDEX IX_EmployeeName  
ON Employees(EmployeeName);
```

Structure

```
EmployeeName
  |
  ▼
Pointer
  |
  ▼
Actual Row
```

Key Rule

Multiple Non-Clustered Indexes Allowed

Example

```
EmployeeName
Email
City
DepartmentID
```

can all have indexes.

Best For

```
WHERE
JOIN
ORDER BY
```

columns.



Unique Index

Prevents duplicate values.

Example


```
CREATE UNIQUE INDEX IX_Email  
ON Employees(Email);
```

Allowed

```
abc@email.com  
xyz@email.com
```

Not Allowed

```
abc@email.com  
abc@email.com
```

Error occurs.

Why Use Unique Index?

Ensures data integrity.

Examples

```
Email  
Username  
Passport Number  
PAN Number
```

Composite Index

Index built on multiple columns.

Example

```
CREATE INDEX IX_Employee  
ON Employees  
(
```

```
    DepartmentID,  
    Salary  
);
```

Useful Query

```
SELECT *  
FROM Employees  
WHERE DepartmentID = 1  
AND Salary > 50000;
```

Order Matters

Good:

```
(DepartmentID, Salary)
```

Query

```
WHERE DepartmentID = 1
```

uses index.

Query

```
WHERE Salary > 50000
```

may not fully use it.

Left-Most Rule

Most databases use:

```
First Column First
```

Creating Indexes

Create Non-Clustered Index

```
CREATE INDEX IX_EmployeeName  
ON Employees(EmployeeName);
```

Create Unique Index

```
CREATE UNIQUE INDEX IX_Email  
ON Employees(Email);
```

Create Composite Index

```
CREATE INDEX IX_DepartmentSalary  
ON Employees  
(  
    DepartmentID,  
    Salary  
);
```

View Existing Indexes

```
sp_helpindex Employees;
```

Dropping Indexes

Remove indexes when no longer needed.

Syntax

```
DROP INDEX IndexName  
ON TableName;
```

Example

```
DROP INDEX IX_EmployeeName  
ON Employees;
```

Important

Only index removed.

Data remains.

10 Index Advantages

Faster Searches

```
WHERE EmployeeID = 100
```

Faster Joins

```
JOIN Employees  
ON DepartmentID
```

Faster Sorting

```
ORDER BY EmployeeName
```

Faster Aggregation

```
GROUP BY DepartmentID
```

Better Dashboard Performance

Large reports execute faster.

Index Disadvantages

Additional Storage

Indexes consume disk space.

Slower INSERT

Example

```
INSERT
```

must update indexes.

Slower UPDATE

Modified values require index updates.

Slower DELETE

Index entries removed too.

Too Many Indexes

Can reduce performance.

Example

Bad:

```
20 indexes  
on  
small table
```

1 2 Index Maintenance

Indexes become fragmented over time.

Why?

Due to:

```
INSERT  
UPDATE  
DELETE
```

operations.

Reorganize

Light maintenance.

```
ALTER INDEX ALL  
ON Employees  
REORGANIZE;
```

Rebuild

Heavy maintenance.

```
ALTER INDEX ALL  
ON Employees  
REBUILD;
```

Monitor Fragmentation

Using SQL Server DMVs.

Advanced topic.

1 **3** Execution Plans

Execution plans show:

```
How SQL Executes Queries
```

Types:

```
Index Seek  
Index Scan  
Table Scan
```

Best

```
Index Seek
```

Worst

Table Scan

on large tables.

Example

Query

```
SELECT *  
FROM Employees  
WHERE EmployeeID = 100;
```

With Index

Index Seek

Without Index

Table Scan

1

4

Best Practices

Index Frequently Searched Columns

Good

```
EmployeeID  
Email  
DepartmentID
```

Index Join Columns

Good

Foreign Keys

Avoid Too Many Indexes

Balance read and write performance.

Use Composite Indexes Carefully

Order columns properly.

Monitor Execution Plans

Verify index usage.

Review Unused Indexes

Remove unnecessary ones.

Professional Example

```
CREATE INDEX IX_Orders_CustomerID  
ON Orders(CustomerID);
```

Reason:

```
JOIN Customers
```

occurs frequently.

1 **5** Common Interview Questions

What is an Index?

Improves query performance.

Difference Between Clustered and Non-Clustered Index?

Clustered:

Sorts actual data.

Non-Clustered:

Stores pointers.

How Many Clustered Indexes?

One

per table.

How Many Non-Clustered Indexes?

Many

Why Can Indexes Slow Down Inserts?

Indexes must also be updated.

What is a Composite Index?

Multiple columns in one index.

What is an Index Seek?

Direct lookup using an index.

1

6

Business Analytics Examples

Employee Search Optimization

```
CREATE INDEX IX_EmployeeID  
ON Employees(EmployeeID);
```

Customer Lookup

```
CREATE INDEX IX_CustomerEmail  
ON Customers(Email);
```

Sales Dashboard

```
CREATE INDEX IX_Sales_Date  
ON Sales(SaleDate);
```

Revenue Analysis

```
CREATE INDEX IX_Department  
ON Employees(DepartmentID);
```

Order Analytics

```
CREATE INDEX IX_OrderDate  
ON Orders(OrderDate);
```

Data Warehouse Example

Fact Tables often contain:

Millions of Rows

Indexes are critical for:

Power BI
Tableau
SSRS

performance.



Summary

In this module, you learned:

- ✓ Index Basics
- ✓ Clustered Index
- ✓ Non-Clustered Index
- ✓ Unique Index
- ✓ Composite Index
- ✓ Creating Indexes
- ✓ Dropping Indexes
- ✓ Execution Plans
- ✓ Index Maintenance
- ✓ Performance Optimization



Practice Questions

Theory

1. What is an Index?
2. Why are Indexes needed?
3. What is a Clustered Index?
4. What is a Non-Clustered Index?
5. How many Clustered Indexes can a table have?
6. What is a Composite Index?

7. What is a Unique Index?
 8. Why can indexes slow down INSERT statements?
 9. What is an Index Seek?
 10. What is a Table Scan?
-

Practical Exercises

Task 1

Create a non-clustered index on EmployeeName.

Task 2

Create a unique index on Email.

Task 3

Create a composite index.

Task 4

Drop an index.

Task 5

Compare query performance:

```
Before Index  
After Index
```

Task 6

View table indexes using:

```
sp_helpindex
```

Challenge Project

Create:

Employees
Customers
Orders
Sales
Products

Add indexes for:

- Primary Keys
- Foreign Keys
- Search Columns
- Reporting Columns

Analyze:

- Query Speed
- Execution Plans
- Index Usage



Next Module

→ **Module 21: Stored Procedures**

Topics Covered:

- What are Stored Procedures?
- Creating Procedures
- Executing Procedures
- Input Parameters
- Output Parameters
- Error Handling
- Performance Benefits
- Real-World Applications